

Projekt: Webserver mit Automatisierung, Backup und Rechteverwaltung

Aufgabe: Webserver-Installation und Bereitstellung einer Webseite

Herzlichen Glückwunsch zur erfolgreichen Weiterbildung zum DevOps Engineer! Ihre erste Aufgabe bei der Firma IT-Boss-Solutions besteht darin, eine einfache Webserver-Lösung zu erstellen. Diese Aufgabe hilft Ihnen, Ihre Fähigkeiten in der Webserver-Konfiguration, Automatisierung und Rechteverwaltung zu vertiefen. Die Aufgabe ist in mehrere Schritte unterteilt, die aufeinander aufbauen.

Ziel:

Bereitstellung einer Webseite der Firma IT-Boss-Solutions auf einem Apache-Webserver. Zudem lernen Sie, wie man den Webserver überwacht und Fehlerbehandlungen mit if-else oder switch-case Strukturen durchführt.

Teil 1: Webserver-Installation und Bereitstellung

Ziel: Installieren Sie einen Apache-Webserver und stellen Sie eine einfache Webseite bereit.

Aufgabenstellung:

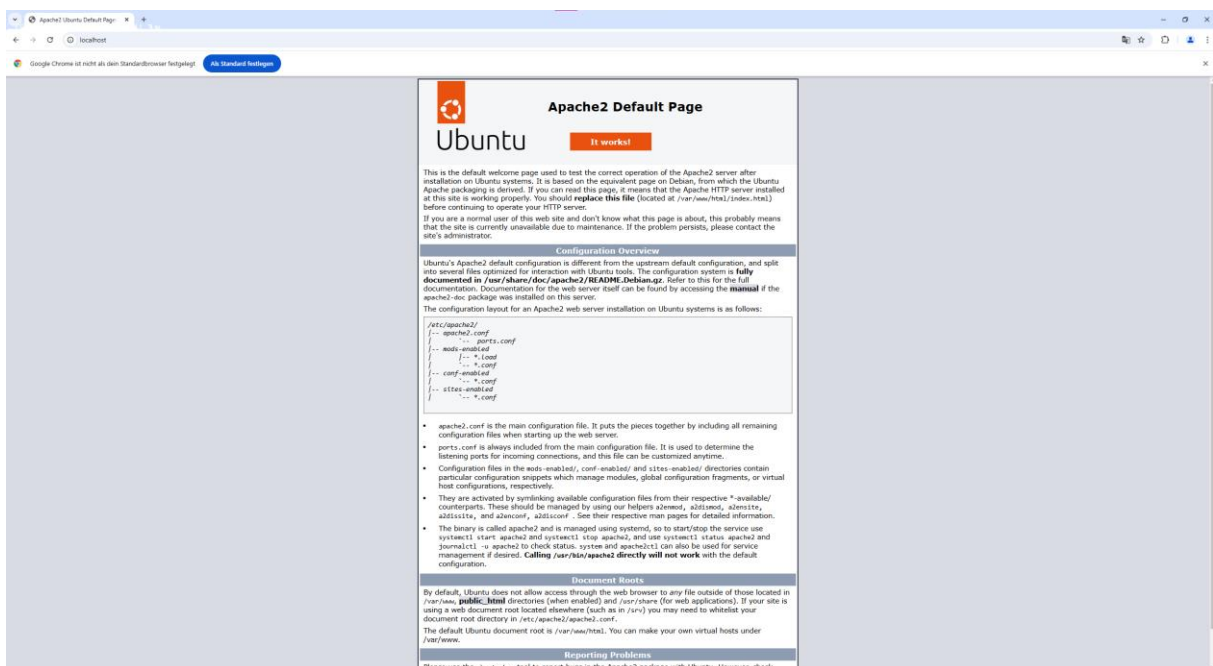
1. Installation des Apache-Webserver

- Verwenden Sie das Paketmanagement-Tool (apt für Ubuntu/Debian oder yum für CentOS/RedHat), um den Apache-Webserver zu installieren.

```
$ projekt30.04.sh
1  #!/bin/bash
2
3  # Apache installieren
4  sudo apt update && sudo apt install apache2 -y
5
6  # Bestätigung
7  echo "Apache wurde erfolgreich installiert!"
8
9  echo "Teil 1 beendet -----"
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Enabling module reqtimeout.
Enabling conf charset.
Enabling conf localized-error-pages.
Enabling conf other-vhosts-access-log.
Enabling conf security.
Enabling conf serve-cgi-bin.
Enabling site 000-default.
Created symlink /etc/systemd/system/multi-user.target.wants/apache2.service → /usr/lib/systemd/system/apache2.service.
Created symlink /etc/systemd/system/multi-user.target.wants/apache-htcacheclean.service → /usr/lib/systemd/system/apache-htcacheclean.service.
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.4) ...
Apache wurde erfolgreich installiert!
Teil 1 beendet -----
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$
```



sudo: führt den Befehl als Administrator aus. Ohne sudo würde der Befehl ohne ausreichende Rechte eventuell fehlschlagen, z. B. bei der Installation von Software.

apt update aktualisiert die Paketquellen, damit wir die aktuellsten Versionen von den Softwares haben.

apt install apache2 installiert das Paket „apache2“, das den Apacheserver enthält. Wir wollen die aktuellste Version von Apache nutzen und die aktuellste Version ist „[2.4.63](#) (released 2025-01-23)“. hier reicht es vollkommen aus, wenn man apache2 schreibt. rest macht Linux selbst.

der Link zum überprüfen von der Version:

<https://httpd.apache.org/download.cgi>

im Browser eingeben:

<http://localhost>

Damit ruft man die Apache-Webseite auf, die unter /var/www/html/index.html liegt.

Apache-Dienst muss aber dafür laufen. wenn nichts kommt, dann:

```
sudo systemctl start apache2
```

Dies brauchen wir damit der Apache Webserver gestartet wird und Webseiten ausgeliefert werden können.

2. Starten und Aktivieren des Webservers

- Starten Sie den Apache-Webserver und konfigurieren Sie ihn so, dass er beim Booten des Servers automatisch startet.

```
11 # Apache so konfigurieren, dass er beim Booten automatisch startet
12 sudo systemctl enable apache2
13
14 # Apache starten
15 sudo systemctl start apache2
16
17 # Bestätigung
18 echo "Apache wurde gestartet und so konfiguriert, dass er beim Booten automatisch startet!"
19
20 echo "Teil 2 beendet -----"
```

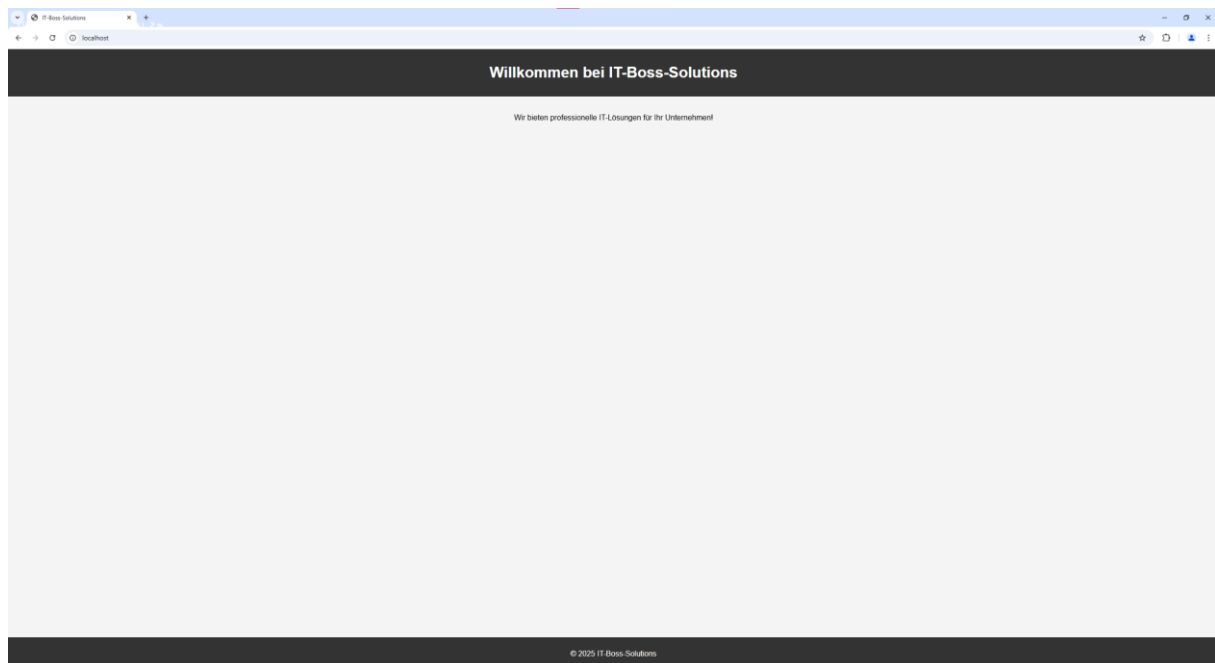
```
Teil 1 beendet -----
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
Apache wurde gestartet und so konfiguriert, dass er beim Booten automatisch startet!
Teil 2 beendet -----
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$
```

Ich habe den Apache-Webserver so konfiguriert, dass er beim Booten des Systems automatisch startet, und ihn anschließend manuell gestartet. Damit stelle ich sicher, dass der Webserver sowohl jetzt als auch nach einem Neustart verfügbar ist. Abschließend habe ich eine Bestätigung auf der Konsole ausgeben lassen.

3. Bereitstellung der Webseite

- Kopieren Sie die bereitgestellten HTML- und CSS-Dateien in das Verzeichnis `/var/www/html/`. Diese Dateien sollten eine einfache Webseite der Firma "IT-Boss-Solutions" darstellen, die den Titel "IT-Boss-Solutions" anzeigt.

```
22 # Bereitstellung der Webseite (HTML und CSS Dateien kopieren)
23 sudo cp /mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04/*.html /var/www/html/
24 sudo cp /mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04/*.css /var/www/html/
25
26 # Bestätigung
27 echo "Webseite wurde bereitgestellt!"
28
29 echo "Teil 3 beendet -----"
```



Apache verwendet standardmäßig `/var/www/html/`, weil es der vordefinierte Ordner für Webinhalte ist. Wenn man HTML- und CSS-Dateien dorthin kopiert, weiß Apache, dass es diese anzeigen soll, wenn jemand `http://localhost` aufruft. Dieser Ordner ist der einzige, auf den Apache Zugriff hat, um Webseiten bereitzustellen.

4. Überprüfung des Webservers

- Stellen Sie sicher, dass der Webserver korrekt läuft, indem Sie die IP-Adresse des Servers in Ihrem Browser aufrufen (z. B. <http://localhost>).
- Wenn der Webserver nicht läuft, soll eine Fehlerbehandlung erfolgen.

Fehlerbehandlung mit if-else:

Verwenden Sie eine if-else-Bedingung, um zu überprüfen, ob der Apache-Webserver erfolgreich gestartet wurde. Falls der Webserver nicht läuft, geben Sie eine Fehlermeldung aus und stoppen Sie das Skript.

```
31  if [ "$(systemctl is-active apache2)" = "active" ]; then
32      echo "Apache läuft, die Webseite ist erreichbar unter http://localhost"
33  else
34      echo "Fehler: Apache läuft nicht!"
35      exit 1
36  fi
37
38  echo "Teil 4 beendet -----"
```

Mit `if [ "$(systemctl is-active apache2)" = "active" ]; then` prüfe ich direkt, ob der Apache-Webserver aktiv ist. Der Befehl `systemctl is-active apache2` gibt den Status des Apache-Dienstes zurück, z. B. `active` oder `inactive`. Wenn Apache läuft, zeigt mein Skript eine Erfolgsmeldung. Wenn nicht, wird eine Fehlermeldung ausgegeben und das Skript wird mit `exit 1` beendet.

```
Teil 2 beendet -----
Webseite wurde bereitgestellt!
Teil 3 beendet -----
Apache läuft, die Webseite ist erreichbar unter http://localhost
Teil 4 beendet -----
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$
```

Teil 2: Automatisierung der Sicherung

Ziel: Erstellen Sie ein einfaches Backup-System, das regelmäßig Sicherungen der Webseitendaten durchführt.

Aufgabenstellung:

1. Erstellen eines Backup-Skripts

- Schreiben Sie ein Bash-Skript, das automatisch eine Sicherung des Verzeichnisses `/var/www/html/` erstellt und diese an einem sicheren Ort speichert (z. B. `/backups/`).

```
$ projektteil2.sh > ...
1  #!/bin/bash
2
3  # Zielordner
4  ZIEL="/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04/backup/webserver_backup_$(date +%Y-%m-%d_%H-%M-%S)"
5
6  # Neues Backup kopieren
7  cp -r /var/www/html "$ZIEL"
8
9  echo "Backup wurde erstellt."
```

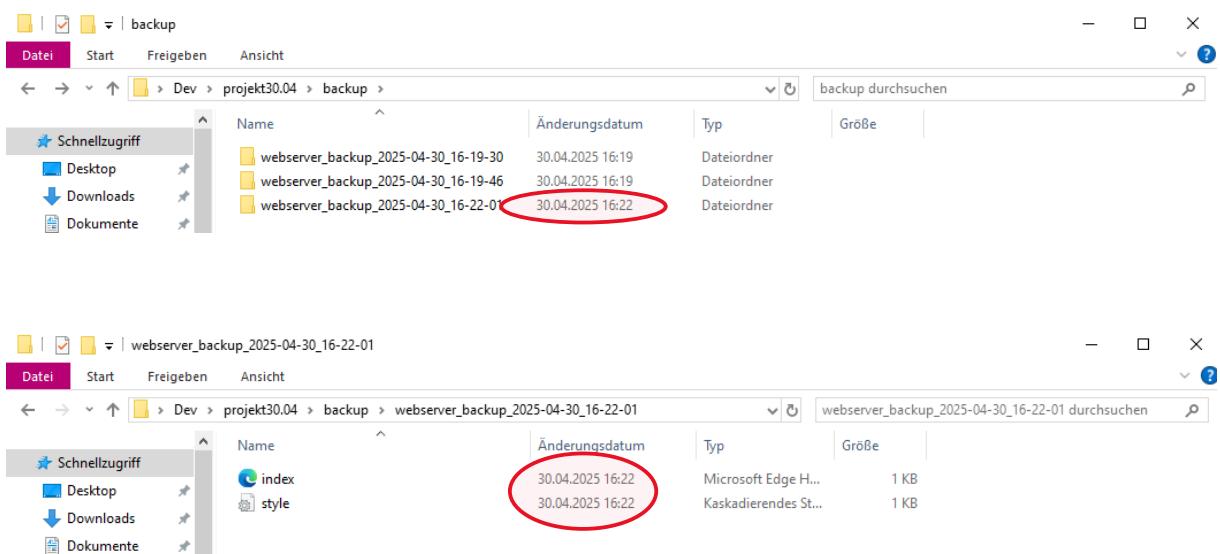
Das Skript startet mit der Bash-Shell und legt in der Variablen ZIEL den Zielordner für das Backup fest. Anschließend wird der gesamte Inhalt des Verzeichnisses `/var/www/html` mit dem Befehl `cp -r` in den festgelegten Zielordner kopiert. Am Ende gibt das Skript eine Bestätigung aus, dass das Backup erfolgreich erstellt wurde.

2. Automatisierung der Backups

- Richten Sie einen Cron-Job ein, der das Backup täglich zur gleichen Zeit (z. B. um 2 Uhr nachts) ausführt.

```
GNU nano 7.2 /tmp/crontab.g
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
22 16 * * * /bin/bash /mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04/projektteil2.sh
```

Um einen Cronjob einzurichten, habe ich `crontab -e` im Terminal eingegeben, um den Cron-Editor zu öffnen. Dann habe ich die gewünschte Ausführungszeit eingegeben, in meinem Fall `22 16 * * *`, was bedeutet, dass der Cronjob täglich um 16:22 Uhr ausgeführt wird. Im nächsten Schritt habe ich den vollständigen Pfad zum Skript angegeben, das ausgeführt werden soll. Der Cronjob ist nun eingerichtet und wird täglich um 16:22 Uhr automatisch ausgeführt.



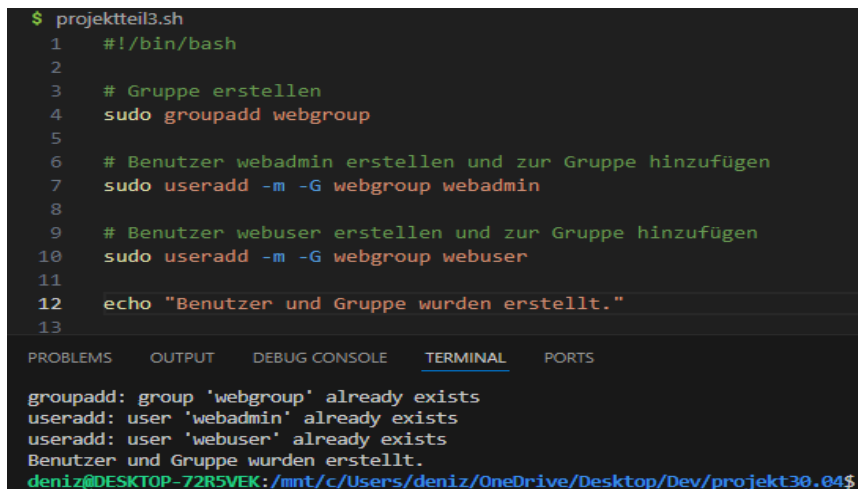
Teil 3: Benutzer- und Gruppenverwaltung

Ziel: Verwalten Sie Benutzer und Gruppen, um den Zugriff auf Server-Ressourcen zu kontrollieren.

Aufgabenstellung:

1. Erstellen von Benutzern und Gruppen

- Erstellen Sie zwei neue Benutzer: webadmin (für die Verwaltung des Webservers) und webuser (für den Zugriff auf die Webseite).
- Erstellen Sie eine Gruppe namens webgroup, der beide Benutzer angehören sollen.



```
$ projektteil3.sh
1  #!/bin/bash
2
3  # Gruppe erstellen
4  sudo groupadd webgroup
5
6  # Benutzer webadmin erstellen und zur Gruppe hinzufügen
7  sudo useradd -m -G webgroup webadmin
8
9  # Benutzer webuser erstellen und zur Gruppe hinzufügen
10 sudo useradd -m -G webgroup webuser
11
12 echo "Benutzer und Gruppe wurden erstellt."
13

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

groupadd: group 'webgroup' already exists
useradd: user 'webadmin' already exists
useradd: user 'webuser' already exists
Benutzer und Gruppe wurden erstellt.
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$
```

„Ich habe das ursprüngliche Skript geändert, um zu vermeiden, dass beim wiederholten Ausführen Fehler wie groupadd: group 'webgroup' already exists und useradd: user 'webadmin' already exists erscheinen. Stattdessen prüft das Skript jetzt, ob die Gruppe webgroup und die Benutzer webadmin und webuser bereits existieren, bevor es versucht, sie zu erstellen. Falls sie existieren, wird eine entsprechende Meldung ausgegeben, andernfalls werden sie erstellt. Diese Änderungen sorgen dafür, dass keine Fehler auftreten und das Skript mehrmals ohne Probleme ausgeführt werden kann.“

Das neue Script:

```
$ projektteil3test.sh
1  #!/bin/bash
2
3  # Prüfen, ob die Gruppe existiert, wenn nicht, dann erstellen
4  if ! getent group webgroup > /dev/null 2>&1; then
5      sudo groupadd webgroup
6      echo "Gruppe webgroup wurde erstellt."
7  else
8      echo "Gruppe webgroup existiert bereits."
9  fi
10
11 # Prüfen, ob der Benutzer webadmin existiert, wenn nicht, dann erstellen
12 if ! id "webadmin" &>/dev/null; then
13     sudo useradd -m -G webgroup webadmin
14     echo "Benutzer webadmin wurde erstellt und in die Gruppe webgroup hinzugefügt."
15 else
16     echo "Benutzer webadmin existiert bereits."
17 fi
18
19 # Prüfen, ob der Benutzer 'webuser' existiert, wenn nicht, dann erstellen
20 if ! id "webuser" &>/dev/null; then
21     sudo useradd -m -G webgroup webuser
22     echo "Benutzer webuser wurde erstellt und in die Gruppe webgroup hinzugefügt."
23 else
24     echo "Benutzer webuser existiert bereits."
25 fi
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Gruppe webgroup existiert bereits.
Benutzer webadmin existiert bereits.
Benutzer webuser existiert bereits.
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04\$

2. Ändern der Berechtigungen

- Ändern Sie die Berechtigungen für das Verzeichnis `/var/www/html/`, sodass nur webadmin Schreibzugriff hat und webuser nur Lesezugriff erhält.

```
29 # Besitzer und Gruppe ändern
30 sudo chown webadmin:webgroup /var/www/html/
31
32 # Berechtigungen setzen
33 sudo chmod 755 /var/www/html/
34 sudo chmod 644 /var/www/html/*
35
36 echo "Berechtigungen und Besitzer wurden angepasst."
37
38 echo "Teil 2 beendet -----"
```

Ich habe zuerst mit `sudo chown webadmin:webgroup /var/www/html/` den Besitzer des Verzeichnisses auf den Benutzer webadmin und die Gruppe webgroup gesetzt. Das bedeutet, webadmin ist jetzt der Hauptverantwortliche für das Verzeichnis, und alle Gruppenmitglieder von webgroup haben auch Zugriff. Danach habe ich mit `chmod 755 /var/www/html/` die Rechte gesetzt: Die erste Zahl (7) steht für den Besitzer und bedeutet „Lesen, Schreiben, Ausführen“ (rwx). Die zweite Zahl (5) ist für die Gruppe und bedeutet „Lesen und Ausführen“ (r-x). Die dritte Zahl (5) ist für alle anderen Benutzer und erlaubt ihnen ebenfalls Lesen und Ausführen. Für die Dateien im Verzeichnis habe ich dann `chmod 644 /var/www/html/*` verwendet. Dabei steht die 6 für den Besitzer (Lesen und Schreiben: rw-), und die 4 jeweils für Gruppe und andere (nur Lesen: r--). So habe ich erreicht, dass nur webadmin die Dateien bearbeiten kann, während webuser (als Mitglied der Gruppe) und alle anderen sie nur lesen dürfen. Am Ende gibt das Skript eine Nachricht aus, dass alles erfolgreich eingestellt wurde.

```
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$ id webadmin; id webuser; getent group webgroup
uid=1001(webadmin) gid=1002(webadmin) groups=1002(webadmin),1001(webgroup)
uid=1002(webuser) gid=1003(webuser) groups=1003(webuser),1001(webgroup)
webgroup:x:1001:webadmin,webuser
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$
```

Die uid=1001 zeigt die ID des Benutzers „webadmin“, gid=1002 ist die ID der Gruppe von webadmin. In groups=1002(webadmin),1001(webgroup) steht, dass webadmin zur Gruppe „webadmin“ (ID 1002) und zur Gruppe „webgroup“ (ID 1001) gehört. Bei webuser ist es genauso: uid=1002, gid=1003 und er gehört ebenfalls zur Gruppe „webgroup“. Der Befehl `getent group webgroup` zeigt, dass die Gruppe „webgroup“ ID 1001 hat und beide Benutzer, ein Mitglied der Gruppe sind.

3. Überprüfen der Berechtigungen

- Überprüfen Sie mit dem `ls -l`-Befehl, ob die Berechtigungen korrekt gesetzt wurden.

```
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$ ls -l /var/www/html/
total 8
-rw-r--r-- 1 webadmin webgroup 503 Apr 23 15:39 index.html
-rw-r--r-- 1 webadmin webgroup 419 Apr 23 15:39 style.css
```

Der Besitzer (webadmin) hat Lese- und Schreibrechte (rw-), die Gruppe (webgroup) hat nur Leserechte (r--), und andere Benutzer auch nur Leserechte (r--). So kann nur der Admin Dateien ändern, alle anderen dürfen sie nur lesen.

Teil 4: Schleifen und reguläre Ausdrücke (Regex)

Ziel: Verwenden Sie Schleifen und reguläre Ausdrücke (Regex), um mehrere Dateien im Webserver-Verzeichnis zu überprüfen und zu benennen.

Aufgabenstellung:

1. Überprüfen der Dateien im Webserver-Verzeichnis

- Schreiben Sie ein Bash-Skript, das alle Dateien im Verzeichnis `/var/www/html/` überprüft und deren Dateinamen mit einem bestimmten Muster (z. B. alle HTML-Dateien) anzeigt.
- Verwenden Sie dabei reguläre Ausdrücke, um nur die Dateien zu filtern, die mit `.html` enden.

2. Verwenden von Schleifen

- Verwenden Sie eine Schleife (for oder while), um alle HTML-Dateien im Verzeichnis `/var/www/html/` zu durchlaufen und deren Namen auszugeben.

```
$ projektteil4.sh
1  #!/bin/bash
2
3  for file in /var/www/html/*; do
4      if [[ $(basename "$file") =~ \.html$ ]]; then
5          echo "Gefundene HTML-Datei: $(basename "$file")"
6      fi
7  done
```

```
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$ bash projektteil4.sh
Gefundene HTML-Datei: index.html
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$
```

Das Skript nutzt eine for-Schleife, um alle Dateien im Verzeichnis `/var/www/html/` zu durchsuchen. Der reguläre Ausdruck `~\.html$` wird verwendet, um nur die Dateien zu filtern, die mit `.html` enden. Für jede Datei wird die `basename`-Funktion verwendet, um den Dateinamen ohne den Pfad anzuzeigen. Diese Dateinamen werden dann mit `echo` ausgegeben, sodass nur die Namen der HTML-Dateien angezeigt werden.

Teil 5: Log-Dateien erstellen und Fehlerprotokollierung

Ziel: Erstellen Sie ein Skript, das Log-Dateien schreibt und Fehlerprotokolle mit Schleifen verwaltet.

Aufgabenstellung:

1. Erstellen eines Log-Datei-Skripts

- Schreiben Sie ein Bash-Skript, das Log-Dateien erstellt und alle relevanten Aktivitäten des Webserver aufzeichnet. Das Skript sollte eine Datei `webserver.log` im Verzeichnis `/var/log/` anlegen und dort Informationen über die Webserver-Aktivitäten protokollieren, wie z. B. Start und Stopp des Servers, Fehler und erfolgreiche Aktionen.

```
$ projektteil55.sh > ...
1  #!/bin/bash
2
3  logfile="/var/log/webserver.log"
4
5  # Teil 1: Start und Stopp protokollieren
6  if systemctl start apache2; then
7      echo "Webserver gestartet" >> "$logfile"
8  else
9      echo "Fehler: Webserver konnte nicht gestartet werden" >> "$logfile"
10 fi
11
12 if systemctl stop apache2; then
13     echo "Webserver gestoppt" >> "$logfile"
14 else
15     echo "Fehler: Webserver konnte nicht gestoppt werden" >> "$logfile"
16 fi
17
18 echo "Teil 1 beendet -----"
19
```

```
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$ cat /var/log/webserver.log
Webserver gestartet
Webserver gestoppt
```

2. Fehlerprotokollierung

- Falls ein Fehler bei einer der Aktionen auftritt (z. B. beim Starten des Webservers oder beim Kopieren von Dateien), soll eine Fehlermeldung in die Log-Datei geschrieben werden.

In Teil 1 wird der Webserver gestartet und gestoppt, und es wird jeweils in eine Log-Datei geschrieben, ob die Aktion erfolgreich war oder nicht. Falls ein Fehler auftritt, wird auch eine Fehlermeldung in die Log-Datei geschrieben. Das bedeutet, dass Teil 2 hier bereits abgedeckt wurde, da bei den Aktionen (Starten und Stoppen des Webservers) Fehlerprotokolle erstellt werden.

3. Verwendung von Schleifen zur Protokollierung

- Erstellen Sie eine Schleife, die die letzten 5 Log-Einträge aus der Log-Datei liest und auf der Konsole anzeigt.

```
21 # Teil 3: Letzte 5 Log-Einträge anzeigen
22 echo "Letzte 5 Log-Einträge:"
23 tail -n 5 "$logfile"
24
25 echo "Teil 3 beendet -----"
```

```
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$ sudo bash projektteil55.sh
Teil 1 und Teil 2 beendet -----
Letzte 5 Log-Einträge:
Webserver gestoppt
Webserver gestartet
Webserver gestoppt
Webserver gestartet
Webserver gestoppt
Teil 3 beendet -----
```

1. Webserver gestoppt
2. Webserver gestartet
3. Webserver gestoppt
4. Webserver gestartet
5. Webserver gestoppt

In Teil 3 des Skripts wird eine Schleife genutzt, um die letzten fünf Log-Einträge aus der Datei `/var/log/webserver.log` anzuzeigen. Der Befehl `tail -n 5` sorgt dafür, dass nur die letzten fünf Zeilen der Log-Datei angezeigt werden. So kann man schnell sehen, was zuletzt im Webserver-Protokoll passiert ist. Nachdem die Einträge angezeigt wurden, wird mit einer Nachricht das Ende des Teils angezeigt.

4. Fehlerbehandlung in der Log-Datei

- Verwenden Sie eine if-else-Struktur, um zu überprüfen, ob eine Aktion erfolgreich war, und protokollieren Sie das Ergebnis. Falls ein Fehler auftritt, schreiben Sie eine entsprechende Fehlermeldung in die Log-Datei.

```
28 # Teil 4: Fehlerbehandlung für Aktionen
29 if ! systemctl start apache2; then
30 |   echo "Fehler: Webserver konnte nicht gestartet werden" >> "$logfile"
31 | else
32 |   echo "Webserver gestartet" >> "$logfile"
33 | fi
34
35 if ! systemctl stop apache2; then
36 |   echo "Fehler: Webserver konnte nicht gestoppt werden" >> "$logfile"
37 | else
38 |   echo "Webserver gestoppt" >> "$logfile"
39 | fi
40 echo "Teil 4 beendet -----"
41
```

```
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$ sudo bash projektteil55.sh
Teil 1 und Teil 2 beendet -----
Letzte 5 Log-Einträge:
Webserver gestartet
Webserver gestoppt
Teil 3 beendet -----
Teil 4 beendet -----
deniz@DESKTOP-72R5VEK:/mnt/c/Users/deniz/OneDrive/Desktop/Dev/projekt30.04$ sudo cat /var/log/webserver.log
Webserver gestartet
Webserver gestoppt
Webserver gestartet
Webserver gestoppt
```

Im vierten Teil des Skripts wurde eine Fehlerbehandlung mit einer if-else-Struktur eingebaut, um sicherzustellen, dass beim Starten und Stoppen des Webserver überprüft wird, ob die Aktionen erfolgreich ausgeführt wurden. Falls eine Aktion fehlschlägt, wird eine Fehlermeldung in die Log-Datei geschrieben, andernfalls eine Bestätigung, dass der Webserver gestartet oder gestoppt wurde, in die Log-Datei protokolliert.

Im Terminal habe ich zuerst die Log-Datei mit `sudo rm /var/log/webserver.log` gelöscht, um eine frische Log-Datei zu haben. Danach habe ich das Skript mit `sudo bash projektteil5.sh` ausgeführt. Das Skript hat dann die entsprechenden Protokolleinträge erstellt, je nachdem, ob der Webserver erfolgreich gestartet und gestoppt wurde. Am Ende des Skripts konnte ich mit dem Befehl `sudo cat /var/log/webserver.log` den Inhalt der Log-Datei überprüfen und sicherstellen, dass alle Einträge korrekt protokolliert wurden, einschließlich der Start- und Stopp-Bestätigungen des Webserver.

Abschlussbericht:

In diesem Projekt wurde der Apache-Webserver installiert, konfiguriert und verschiedene Automatisierungen sowie Aufgaben zur Fehlerbehandlung durchgeführt.

Teil 1: Installation und Verwaltung des Apache-Webservers

Zuerst wurde der Apache-Webserver mit den Befehlen `apt update` und `apt install apache2` installiert. Danach wurde der Webserver gestartet und geprüft, ob er läuft. Über das Terminal konnte ich die Webseite im Browser aufrufen, indem ich „`http://localhost`“ eingab. Falls der Server nicht lief, wurde er mit `sudo systemctl start apache2` gestartet. Die Webseite mit dem Titel „IT-Boss-Solutions“ wurde anschließend ins Verzeichnis `/var/www/html/` kopiert.

Teil 2: Automatisierung der Sicherung

Ein Bash-Skript wurde erstellt, das täglich ein Backup des Webserver-Verzeichnisses `/var/www/html/` anlegt. Um sicherzustellen, dass das Backup regelmäßig ausgeführt wird, richtete ich einen Cron-Job ein. Der Cron-Job sorgt dafür, dass das Skript jeden Tag um 15:45 Uhr nachts automatisch startet.

Teil 3: Benutzer- und Gruppenverwaltung

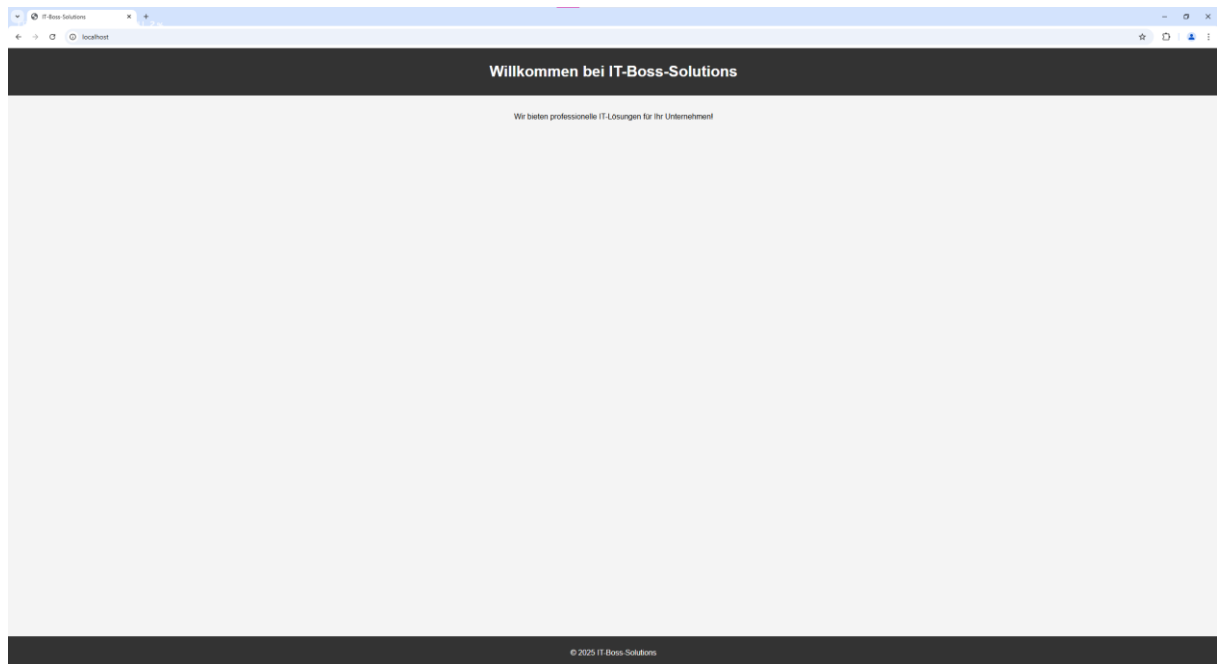
Für den Webserver erstellte ich zwei Benutzer (`webadmin` und `webuser`) sowie eine Gruppe (`webgroup`). Anschließend wurden die Berechtigungen so angepasst, dass nur der Benutzer `webadmin` Dateien im Webserver-Verzeichnis bearbeiten kann, während `webuser` nur Lesezugriff hat. Das Skript prüfte, ob die Benutzer und die Gruppe bereits existieren, um Fehler zu vermeiden.

Teil 4: Schleifen und Reguläre Ausdrücke

Ein weiteres Skript durchsuchte das Webserver-Verzeichnis nach HTML-Dateien und gab die Namen dieser Dateien aus. Dazu wurde eine Schleife verwendet, und mit einem regulären Ausdruck wurde sichergestellt, dass nur Dateien angezeigt werden, die auf „`.html`“ enden.

Teil 5: Log-Dateien und Fehlerprotokollierung

In diesem Teil wurde eine Log-Datei erstellt, die wichtige Serveraktionen wie das Starten und Stoppen des Webserver aufzeichnet. Wenn ein Fehler auftrat, wurde dieser ebenfalls in der Log-Datei festgehalten. Zusätzlich wurde eine Schleife eingebaut, die die letzten fünf Log-Einträge anzeigt, um einen schnellen Überblick zu bekommen, was auf dem Server passiert.



Abschluss:

Nachdem Sie alle Aufgaben erfolgreich erledigt haben, testen Sie die Funktionen Ihres Webservers und stellen Sie sicher, dass die Webseite wie gewünscht angezeigt wird. Überprüfen Sie auch, ob die Log-Dateien korrekt geschrieben und Fehler ordnungsgemäß protokolliert wurden. Dokumentieren Sie Ihre Ergebnisse und reflektieren Sie darüber, wie Sie die Aufgabe noch weiter optimieren können.

Viel Erfolg bei der Aufgabe!

Abgabe ist 30.04.2025

Bereitgestellte Dateien:

Die Webseite von IT-Boss-Solutions besteht aus den folgenden Dateien:

index.html (HTML-Datei der Webseite)

style.css (CSS-Datei für das Design)